

NB02: Regular Languages

Language Processing

LEI/2025-26

Nuno Macedo

Universidade do Minho

(updated 16/02/2026)

2.1 Regular Languages

- Regular languages is the simplest class of languages in Chomsky hierarchy
- They cannot express dependencies like "the same number of symbols" or properly nested structures
- Recognizable by **finite automata**
- Described formally by **regular expressions** (REs)
- Important for **lexical analysis**, input validation, and simple pattern recognition

Example: A Language of Valid Identifiers

- The language of Python identifiers
 -  `aux_function`, `_hiddenVar`, `xy23`
 -  `23xy`, `ab+xy`, `-xyz`

```
aux_function = True
_hiddenVar = True
xy23 = True
# 23xy = True
# ab+xy = True
# -xyz = True
```

- Pattern description:
 - *Arbitrary-length* sequences of *alphanumeric* or *underscore* symbols
 - *Cannot start* with a numeric symbol (why?)
- How can we **formally specify** these rules?
- How can we **efficiently recognize** such identifiers?

2.2 Regular Expressions

- **Regular expressions** (REs), also known as *regexes*, are human-friendly ways to describe regular languages
- Main uses:
 - Define lexing rules for compilers/interpreters
 - Detect patterns in text processing
 - Search and replace in editors or IDEs

Regular expressions in theory

- A RE is built from:
 - **Base cases:**
 - Symbols $a \in \Sigma$
 - Empty set: $\emptyset$
 - Empty word: ϵ
 - **Inductive cases** (for expressions e_1, e_2):

- Concatenation: $e_1 e_2$
- Union / alternation: $e_1 | e_2$
- Kleene closure: e_1^* (zero or more repetitions)

Example: A Language of Valid Identifiers (basic RE)

- $(a|b|\dots|w|z|A|B|\dots|W|Z|_)(a|b|\dots|w|z|A|B|\dots|W|Z|_0|1|\dots|9)^*$
 - $(a|b|\dots|w|z|A|B|\dots|W|Z|_)$ any alphabetic symbol or an underscore
 - $(a|b|\dots|w|z|A|B|\dots|W|Z|_0|1|\dots|9)$ any alpha-numeric symbol or an underscore
 - $(a|b|\dots|w|z|A|B|\dots|W|Z|_0|1|\dots|9)^*$ any alpha-numeric symbol or an underscore, repeated an arbitrary number of times
 - $(a|b|\dots|w|z|A|B|\dots|W|Z|_)(a|b|\dots|w|z|A|B|\dots|W|Z|_0|1|\dots|9)^*$ any alpha-numeric symbol or an underscore, followed by an arbitrary number of alpha-numeric symbols or underscores

Concrete syntax for regular expressions

- Theory of computation deals with the formal notion of REs
- RE processors introduce other operators to facilitate the writing of REs
 - In most cases, syntactic sugar that does not increase theoretical expressivity
- Syntax may vary slightly depending on the RE processor
 - A popular syntax is the one inspired by Perl, which is followed by Python
- Special symbols
 - `.` matches any symbol except a new line
 - `^` matches the start of the string
 - `$` matches the end of the string
- Repetition
 - `*` matches any repetition of the preceeding RE
 - `+` matches one or more repetitions of the preceeding RE
 - `?` matches 0 or 1 repetitions of the preceeding RE
 - `{n}` matches exactly `n` repetitions of the preceeding RE
 - `{m,n}` matches between `n` and `m` repetitions of the preceeding RE
- `|` union (alternative) of two REs
- `[...]` creates a group of alternative symbols
 - `-` can be used for ranges of symbols (to match `-`, put it in the begining or end of group)
 - `^` denotes the complement, all symbols except those declared
 - All symbols treated as literal inside group except `-`, `]` and `^`
- Symbol groups
 - `\d` for any digit symbol
 - `\s` for any whitespace symbol
 - `\w` any alpha-numeric symbol plus the underscore `_`
 - `\b` matches the boundary of words without consuming any symbol
 - Upper-case versions for complement set (e.g., `\D` for any symbol that is **not** a digit)
- Operator precedence is (roughly):
 - Repetition operators $\rightarrow$ Concatenation $\rightarrow$ Alternative
 - Parenthesis `( )` to group REs and change operator precedence
- From now on, we will use this concrete syntax, not the formal RE notation

Example: A Language of Valid Identifiers (extended RE)

- `[a-zA-Z_][a-zA-Z0-9_]*`
 - `a-z` all symbols between `a` and `z`
 - `A-Z` all symbols between `A` and `Z`

- `[0-9]` all symbols between `0` and `9`
- `[a-zA-Z_]` any symbol between `a` and `z`, `A` and `Z` or underscore `_`
- `[a-zA-Z0-9_]` any symbol between `a` and `z`, `A` and `Z`, `0` and `9` or underscore `_`
- `[a-zA-Z0-9_]*` any repetitions of symbols between `a` and `z`, `A` and `Z`, `0` and `9` or underscore `_`
- Or simply `[a-zA-Z_]\w*`

✓ Example: A Language for Floating Point Numbers

```
print(10.34)
print(.34)
print(10.)
# print(.)
print(-.34)
print(+10.)
print(10.34E-3)
print(.34e3)
```

```
10.34
0.34
10.0
-0.34
10.0
0.01034
340.0
```

- Unsigned integer numbers:
`\d+` → `23`
- Unsigned floating point numbers:
`\d+\.\d+` → `23.45`
- Unsigned integer or floating point numbers:
`\d+(\.\d+)?` → `23` or `23.45`
- Unsigned integer or floating point numbers, possibly omitting one side of point but not both:
`\d+(\.\d*)?|\.\d+` → `23` or `23.45` or `.45` or `23.`
- Possibly signed integer or floating point numbers, possibly omitting one side of point but not both:
`[+-]?(\d+(\.\d*)?|\.\d+)` → `+23` or `-23.45` or `.45` or `+23.`
- Possibly signed floating point numbers, possibly omitting one side of point but not both, with optional, possibly signed, exponent:
`[+-]?(\d+(\.\d*|\d*\.\d+)([eE][+-]?\d+)?)` → `+23` or `-23.45E+56` or `.45e-56` or `+23.e56`

Beyond word recognition

- Although REs in theory are used to recognize words belonging to a language, in practice they are often used find substrings in a larger text
- In this case, the output is no longer yes/no, but a set of **matches**
- The language accepted by the RE does not change, but the set of returned matches may change depending on the RE processor
- This is the basis of the tokenization stage in the language processing pipeline

✓ Example: A Language of Valid Identifiers (matches)

- How many identifiers are there in the following code?
- That is, how many matches are there for `[a-zA-Z_]\w*`?

```
for i in range(10): print(i)
```

- The RE will recognize `i`, `range`, `print`, `for` and `in`
- But also `r`, `ra`, `ran`, `rang` and `p`, `pr`, `pri`, `prin`!

✚ `for` and `in` are not actually identifiers, they are *keywords*; we will address this overlap later

Greedy vs. Lazy matching

- Theoretical REs are only concerned about recognition
 - Whether a word belongs to the language or not
- But the same word may be matched by a RE in different ways
 - This is not irrelevant for the user when processing a text
- By default, repetition operators are *greedy*, the match as many symbols as possible
- *Lazy* variants are used when a `?` is appended (`*?`, `+?`, `{n,m}?`)
- This affects how the RE processor matches, but not the overall language
- *Example*: detecting text between quotes in input text `I say "Hello" and you say "World"`
 - Regex `".*"` is greedy, it has a single match, `"Hello" and you say "World"`
 - Regex `".*?"` is lazy, it has two matches, both `"Hello"` and `"World"`

Capturing groups

- Often we are only interested in part of the substring matched by an RE
- REs support the introduction of *capturing groups* using parenthesis `(...)`, that specify what is returned in a match
- *Example*: detecting text between quotes in input text `I say "Hello" and you say "World"`
 - with `".*?"`, matches will include the quotes, `"Hello"` and `"World"`
 - with capturing group `"(.*)"`, matches will be only `Hello` and `World`
- Sometimes we need to use parenthesis to group expressions but do not want a capturing group
 - In that case we have to escape the delimiters with `(?:...)`

Backreferences

- Modern REs also allow *backreferences* to captured groups with `\number`
- This is a powerful feature that goes beyond what theoretical REs allow
- *Example*: find all repeated alpha-numeric patterns
 - `(\w+)\1` matches words like `AA`, `ABAB` or `ABCABC`
- Also useful for text substitution that preserves part of the original match

Exercise: Find title patterns in JSON file

Open file `cinema.json` in any text editor of your choice.

- Find all occurrences of:
 - All titles
 - All titles that include a number
 - All titles that include a **real** number
 - All titles that include **two** numbers
 - All titles where character `x` **does not** occur
 - All titles with **repeated** words
 - All **multi-line** titles
- Remove all subtitles (identified by `:`)
- Surround numbers in titles by single quotes `'`

Regular expressions in Python

- Regular expressions processing in Python are provided by module `re` (see [documentation](#))
- Very rich module with advanced features, based on the well-known Perl syntax



Functions for detecting matches

- `re.match(regex, text)` tries to match `regex` from the beginning of `text`
- `re.fullmatch(regex, text)` tries to match `regex` from the beginning to the end `text`
- `re.search(regex, text)` tries to match `regex` anywhere in `text` (will return first occurrence)
- `re.finditer(regex, text)` returns an iterator over all (non-overlapping) matches of `regex` in `text`
- `re.findall(regex, text)` returns all (non-overlapping) substrings of `text` matched by `regex`
- All these functions, except `findall()`, return `Match` objects with information about the matched substring
- For a `Match` `m`:
 - `m.group()` retrieves the matched substring
 - `m.span()` (or `m.start()` and `m.end()`) returns indexes matched by the substring in the full string

Function for substitutions

- `re.sub(regex, repl, text)` replaces all matches of `regex` in `text` by `repl`
- `repl` can be:
 - a string that can refer to the `regex` capturing groups
 - a function that processes `Match` objects

Raw strings and Python

- Regular expressions have their own special characters prefixed with `\`
- To avoid clashes with Python's special characters, you should write them as raw strings, prefixed with `r`
- `r"\w+"` vs. `"\\w+"`



2.3 Finite Automata

- REs provide a way to specify regular languages
- However, we need a mechanism to operationalize the recognition of words
- *Finite automata* (FA) are such a mechanism for regular languages
- An FA is essentially a state machine with transitions triggered by alphabet symbols
 - After processing the full input, the reached state determines acceptance
- FAs give an answer about language membership in *linear time* over the length of the input
 - Each symbol of the input is inspected once



Throughout the notebook, the `automata` Python module is used to draw and manipulate automata. This is not part of the syllabus.

```
# libraries to allow the visualization and manipulation of automata
!apt-get update -qq
!apt-get install -y -qq graphviz graphviz-dev
!pip install -q pygraphviz coloraide
!pip install -q automata-lib
```

[Mostrar saída oculta](#)

```
from automata.fa.dfa import DFA
from automata.fa.nfa import NFA
```



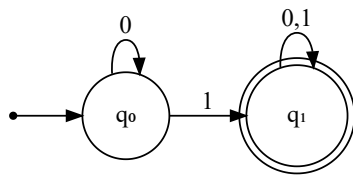
Deterministic Finite Automata



Finite automata by example

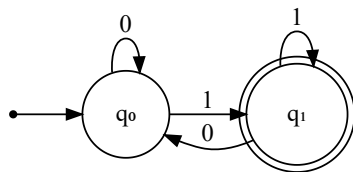
For the alphabet $\Sigma = \{0, 1\}$, what are the languages recognized by the following automata?

```
DFA(
    allow_partial = False,
    states={'q0', 'q1'},
    input_symbols={'0', '1'},
    transitions={
        'q0': { '0': 'q0', '1' : 'q1' },
        'q1': { '0': 'q1', '1' : 'q1' } },
    initial_state='q0',
    final_states={'q1'} ).show_diagram(path=None)
```



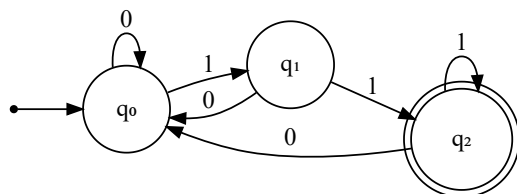
- ☒ 1, 01, 010, ...
- ☒ ϵ , 0, 00, 000, ...
- Bitstrings with at least one 1, or $0^*1(0|1)^*$.

```
DFA(
    allow_partial = False,
    states={'q0', 'q1'},
    input_symbols={'0', '1'},
    transitions={
        'q0': { '0': 'q0', '1' : 'q1' },
        'q1': { '0': 'q0', '1' : 'q1' } },
    initial_state='q0',
    final_states={'q1'} ).show_diagram(path=None)
```



- ☒ 1, 01, 0101, ...
- ☒ ϵ , 0, 10, 1110, ...
- Bitstrings that end in a 1, or $(0|1)^*1$.

```
DFA(
    allow_partial = False,
    states={'q0', 'q1', 'q2'},
    input_symbols={'0', '1'},
    transitions={
        'q0': { '0': 'q0', '1' : 'q1' },
        'q1': { '0': 'q0', '1' : 'q2' },
        'q2': { '0': 'q0', '1' : 'q2' } },
    initial_state='q0',
    final_states={'q2'} ).show_diagram(path=None)
```



- ☒ 11, 011, 01011, ...
- ☒ ϵ , 0, 1, 01, 110, ...
- Bitstrings that end in two 1s, or $(0|1)^*11$.

DFA formal definition

- A deterministic finite automaton (DFA) is a tuple $(Q, \Sigma, \delta, q_i, F)$
 - Q is a finite set of states

- Σ is the input alphabet
- $\delta : Q \times \Sigma \rightarrow Q$ is a transition function
- $q_i \in Q$ is the initial state
- $F \subseteq Q$ is a set of final (or accepting) states
- A word $w \in \Sigma^*$ is processed by a DFA by successively applying δ to the current state and symbols of w , starting in state q_i
 - We call this the extended transition function $\hat{\delta} : Q \times \Sigma^* \rightarrow Q$
 - $\hat{\delta}(q, \varepsilon) = q$
 - $\hat{\delta}(q, xa) = \delta(\hat{\delta}(q, x), a)$
- If the state reached after processing all symbols of w is final, w belongs to the language; otherwise, it does not
 - A word w is accepted iff $\hat{\delta}(q_i, w) \in F$

Finite automata representations

- FAs have two typical, human-friendly representations
- As a *graph*, states are nodes, transitions are edges, initial state with incoming arrow, accepting states with double border (alphabet is implicit)
- As a *table*, states are rows, alphabet symbols are columns, transitions are table entries, initial state with $\rightarrow$, accepting states with $*$

Example: Bitstrings with at least one 1 revisited

$0^*1(0|1)^*$

- Formally
 - $Q = \{q_0, q_1\}$
 - $\Sigma = \{0, 1\}$
 - $q_i = q_0$
 - $F = \{q_1\}$
 - $\delta(q_0, 0) = q_0, \delta(q_0, 1) = q_1, \delta(q_1, 0) = q_1, \delta(q_1, 1) = q_1$
- Table representation

state	0	1
$\rightarrow q_0$	q_0	q_1
$* q_1$	q_1	q_1
- Is 010 accepted?
- Following the table or diagram:
 - start at q_0
 - consume 0, stay in q_0
 - consume 1, transition to q_1
 - consume 0, stay in q_1
 - no more symbols to consume, q_1 is accepting state, word is accepted

$q_0 \xrightarrow{0} q_0 \xrightarrow{1} q_1 \xrightarrow{0} q_1$ ✓

- Or formally:
 - $\hat{\delta}(q_0, 010) = \hat{\delta}(\delta(q_0, 0), 10) = \hat{\delta}(q_0, 10)$
 - $\hat{\delta}(q_0, 10) = \hat{\delta}(\delta(q_0, 1), 0) = \hat{\delta}(q_1, 0)$
 - $\hat{\delta}(q_1, 0) = \hat{\delta}(\delta(q_1, 0), \varepsilon) = \hat{\delta}(q_1, \varepsilon)$
 - $\delta(q_1, \varepsilon) = q_1$
 - $q_1 \in F$ ✓

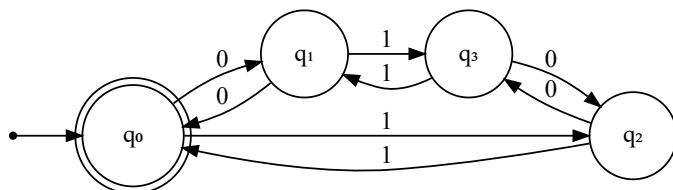
Interpretation of FAs

- Finite automata do not have any memory beyond the (finite) states
- In a sense, each state "memorizes" the paths that have led to it
- Since the number of states is finite by definition, there is a limit to what an FA can remember
 - This limits the languages that FAs can recognize

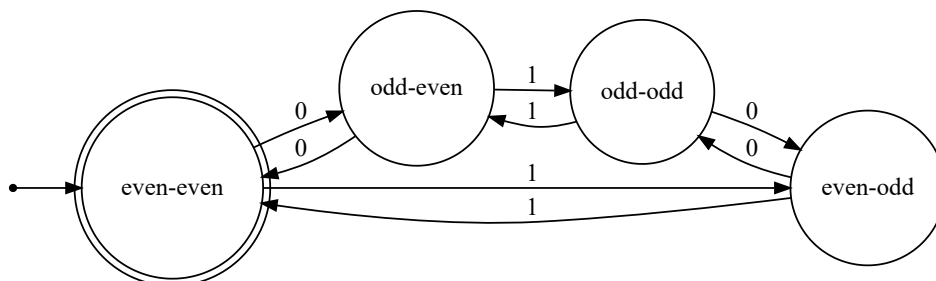
Exercise

- What is the language recognized by the following DFA?
- What do the states represent?
- You can assign any labels to states to make them more intelligible

```
DFA(  
    allow_partial = False,  
    states={'q0', 'q1', 'q2', 'q3'},  
    input_symbols={'0', '1'},  
    transitions={  
        'q0': { '0': 'q1', '1': 'q2' },  
        'q1': { '0': 'q0', '1': 'q3' },  
        'q2': { '0': 'q3', '1': 'q0' },  
        'q3': { '0': 'q2', '1': 'q1' } },  
    initial_state='q0',  
    final_states={'q0'} ).show_diagram(path=None)
```



```
DFA(  
    allow_partial = False,  
    states={'even-even', 'odd-even', 'even-odd', 'odd-odd'},  
    input_symbols={'0', '1'},  
    transitions={  
        'even-even': { '0': 'odd-even', '1': 'even-odd' },  
        'odd-even': { '0': 'even-even', '1': 'odd-odd' },  
        'even-odd': { '0': 'odd-odd', '1': 'even-even' },  
        'odd-odd': { '0': 'even-odd', '1': 'odd-even' } },  
    initial_state='even-even',  
    final_states={'even-even'} ).show_diagram(path=None)
```



Complete vs. incomplete DFAs

- The DFAs we have seen so far are said to be *complete*, because δ is defined for all pairs of states/symbols
- A DFA is said to be *incomplete* if δ is not defined for some pair of states/symbols
- When a non-existing transition is triggered, a word is immediately rejected
- Any incomplete DFA can be made complete trivially by adding a "dead state" that leads nowhere
 - By convention, the "dead state" is denoted $\emptyset$

Example: Complete vs. complete DFA

- All bitstrings that do not have two consecutive 1 symbols
- (What would be the regular expression?)

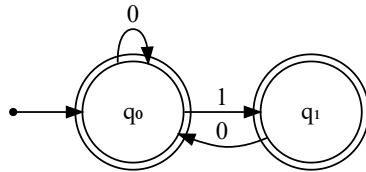
```
dfa = DFA(  
    allow_partial = True,  
    states={'q0', 'q1'},  
    input_symbols={'0', '1'},  
    transitions={  
        'q0': { '0': 'q0', '1': 'q1' },
```



```

'q1': { '0': 'q0' } },
initial_state='q0',
final_states={'q0', 'q1' } )
dfa.show_diagram(path=None)

```

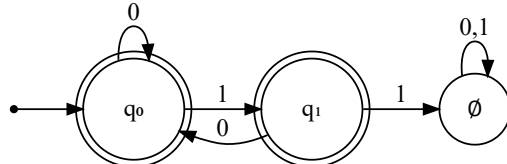


q₀ $\xrightarrow{0}$ q₀ $\xrightarrow{1}$ q₁ $\xrightarrow{0}$ q₀

```

dfa.to_complete("0").show_diagram(path=None)

```



q₀ $\xrightarrow{0}$ q₀ $\xrightarrow{1}$ q₁ $\xrightarrow{1}$ ∅

Example: A Language of Valid Identifiers (FA)

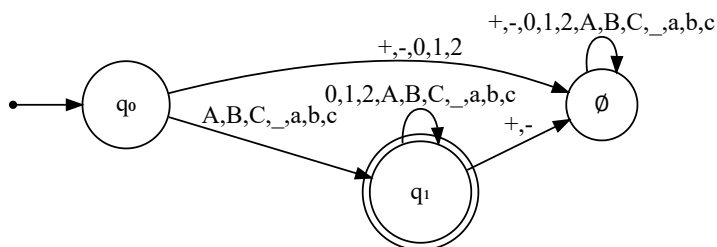
- Simplified version for a reduced alphabet Σ

state	a	b	c	A	B	C	0	1	2	_	+	-
→ q ₀	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁
*q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁	q ₁
q ₂	q ₂	q ₂	q ₂	q ₂	q ₂	q ₂	q ₂	q ₂	q ₂	q ₂	q ₂	q ₂

```

DFA(
  allow_partial = False,
  states={'q0', 'q1', '∅'},
  input_symbols={'a','b','c','A','B','C','0','1','2','_','-','+'},
  transitions={
    'q0': { c: 'q1' for c in {'a','b','c','A','B','C','_'} } | { c: '∅' for c in {'0','1','2','-','+'} },
    'q1': { c: 'q1' for c in {'a','b','c','A','B','C','0','1','2','_'} } | { c: '∅' for c in {'-','+'} },
    '∅': { c: '∅' for c in {'a','b','c','A','B','C','0','1','2','-','+'} },
  },
  initial_state='q0',
  final_states={'q1'} ).show_diagram(path=None)

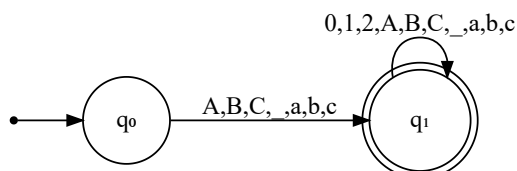
```



```

DFA(
  allow_partial = True,
  states={'q0', 'q1'},
  input_symbols={'a','b','c','A','B','C','0','1','2','_','-','+'},
  transitions={
    'q0': { c: 'q1' for c in {'a','b','c','A','B','C','_'} },
    'q1': { c: 'q1' for c in {'a','b','c','A','B','C','0','1','2','_'} } },
  initial_state='q0',
  final_states={'q1'} ).show_diagram(path=None)

```



✓ Non-deterministic finite automata

- Non-deterministic finite automata allow multiple outgoing transitions for each pair state/symbol
- A non-deterministic finite automaton (NFA) is a tuple $(Q, \Sigma, \delta, q_i, F)$
 - Q is a finite set of states
 - Σ is the input alphabet
 - $\delta : Q \times \Sigma \rightarrow \mathcal{P}(Q)$ is a transition function
 - $q_i \in Q$ is the initial state
 - $F \subseteq Q$ is a set of final (or accepting) states
- When executing an NFA, we can be in multiple states simultaneously, rather than in a single one as in a DFA
- A word $w \in \Sigma^*$ is processed by an NFA by successively applying δ to all the current states and symbols of w , starting in states $\{q_i\}$
 - We call this the extended transition function $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$
 - $\hat{\delta}(q, \varepsilon) = \{q\}$
 - $\hat{\delta}(q, xa) = \bigcup_{p \in \hat{\delta}(q, x)} \delta(p, a)$
- If one of the states reached after processing all symbols of w is final, w belongs to the language; otherwise, it does not
 - A word w is accepted iff $\hat{\delta}(q_i, w) \cap F \neq \emptyset$

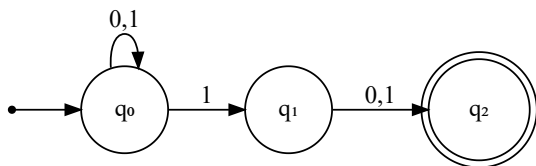
Executing NFAs

- Since we now may be in multiple states of the NFA simultaneously, an execution is a *tree* of states
- This is called a *computation tree*
- A word is accepted if **any** branch of a tree finishes in an accepting state

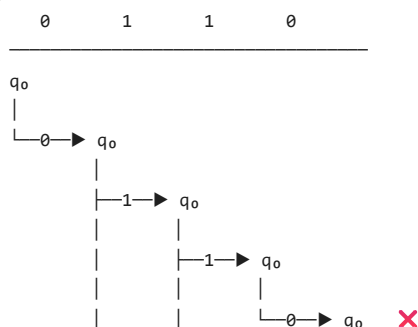
✓ Example: Bitstrings whose 2nd to last symbol is 1

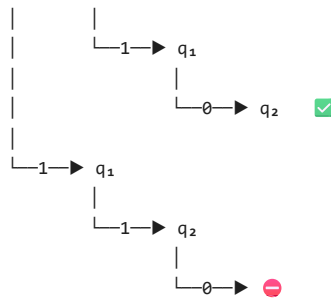
- $(\emptyset|1)^*1(\emptyset|1)$

```
nfa = NFA(
    states={'q0', 'q1', 'q2'},
    input_symbols={'0', '1'},
    transitions={
        'q0': { '0' : {'q0'}, '1' : {'q0', 'q1'} },
        'q1': { '0' : {'q2'}, '1' : {'q2'} },
        'q2': { '0' : {}, '1' : {} } },
    initial_state='q0',
    final_states={'q2'} )
nfa.show_diagram(path=None)
```



- Is $\emptyset 1 1 \emptyset$ accepted by this NFA?
- With a computation tree





- Or formally:

- $\hat{\delta}(q_0, 0110) = \bigcup_{p \in \delta(q_0, 0)} \hat{\delta}(p, 110) = \hat{\delta}(q_0, 110)$
- $\hat{\delta}(q_0, 110) = \bigcup_{p \in \delta(q_0, 1)} \hat{\delta}(p, 10) = \hat{\delta}(q_0, 10) \cup \hat{\delta}(q_1, 10)$
- $\hat{\delta}(q_0, 10) \cup \hat{\delta}(q_1, 10) = (\hat{\delta}(q_0, 0) \cup \hat{\delta}(q_1, 0)) \cup \hat{\delta}(q_2, 0) = (\hat{\delta}(q_0, \varepsilon) \cup \hat{\delta}(q_2, \varepsilon)) \cup \emptyset$
- $\hat{\delta}(q_0, \varepsilon) \cup \hat{\delta}(q_2, \varepsilon) = \{q_0, q_2\}$
- $F \cap \{q_0, q_2\} = \{q_2\} \neq \emptyset$ ✓

NFAs vs. DFAs

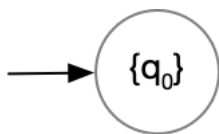
- NFAs and DFAs are **equivalent** formalisms
 - They can recognize exactly the same languages, and can be converted between one another automatically
- Some languages are easier to represent in NFAs
- But DFA recognition is more efficient
- Usually, we manipulate NFAs, but then translate into DFAs for efficient implementation
- However, the resulting DFA can be considerably bigger, with up to $2^{|Q|}$ states for an NFA with states Q

NFA → DFA: subset construction

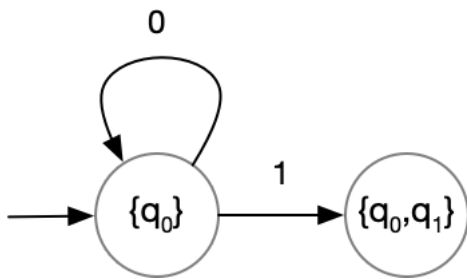
- The core idea behind this construction is to create a state in the DFA for each possible subset of states in the NFA
- We will use these subsets as labels of the DFA to make the transformation clear
 - For instance, a DFA state $\{q_0, q_2\}$ will represent being simultaneously in q_0 and q_2 in the NFA
- To translate an NFA $(Q_N, \Sigma, \delta_N, q_N, F_N)$ into a DFA $(Q_D, \Sigma, \delta_D, q_D, F_D)$, the algorithm is the following:
 - Create all $Q_D = \mathcal{P}(Q_N)$
 - For each subset state $p \in Q_D$ and for each symbol $a \in \Sigma$:
 - $\delta_D(p, a) = \bigcup_{q \in p} \delta_N(q, a)$
 - The initial state q_D is $\{q_N\}$ (it starts in exactly one state, the initial state of the NFA)
 - The final states F_D are all subset states $p \in Q_D$ such that $p \cap F_N \neq \emptyset$ (any state that is in some accepting state of the NFA)
- In practice, it's not worth it to create all states, just create them as needed starting from the initial state $\{q_N\}$

✓ Example: Bitstrings whose 2nd to last symbol is 1 (revisited)

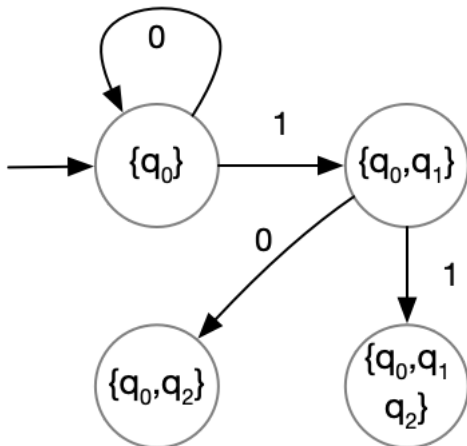
- Let us convert the given NFA into a DFA
- The initial state is $\{q_0\}$



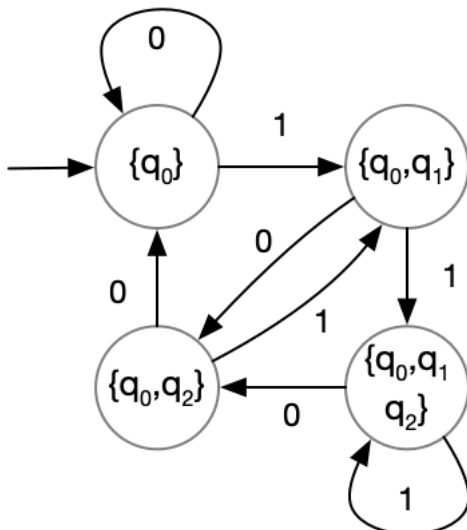
- State $\{q_0\}$ represents a single NFA state, so it is easy:
 - $\delta_N(q_0, 0) = \{q_0\}$ and $\delta_N(q_0, 1) = \{q_0, q_1\}$, so $\delta_D(\{q_0\}, 0) = \{q_0\}$ and $\delta_D(\{q_0\}, 1) = \{q_0, q_1\}$



- This introduced a new state $\{q_0, q_1\}$, which represents two simultaneous states of the NFA; we must apply the transitions to all states
 - $\delta_N(q_0, 0) = \{q_0\}$ and $\delta_N(q_1, 0) = \{q_2\}$, so $\delta_D(\{q_0, q_1\}, 0) = \{q_0, q_2\}$
 - $\delta_N(q_0, 1) = \{q_0, q_1\}$ and $\delta_N(q_1, 1) = \{q_2\}$, so $\delta_D(\{q_0, q_1\}, 1) = \{q_0, q_1, q_2\}$



- This introduced two new states, $\{q_0, q_2\}$ and $\{q_0, q_1, q_2\}$
 - $\delta_N(q_0, 0) = \{q_0\}$ and $\delta_N(q_2, 0) = \emptyset$, so $\delta_D(\{q_0, q_2\}, 0) = \{q_0\}$
 - $\delta_N(q_0, 1) = \{q_0, q_1\}$ and $\delta_N(q_2, 1) = \emptyset$, so $\delta_D(\{q_0, q_2\}, 1) = \{q_0, q_1\}$
 - $\delta_N(q_0, 0) = \{q_0\}$, $\delta_N(q_1, 0) = \{q_2\}$ and $\delta_N(q_2, 0) = \emptyset$, so $\delta_D(\{q_0, q_1, q_2\}, 0) = \{q_0, q_2\}$
 - $\delta_N(q_0, 1) = \{q_0, q_1\}$, $\delta_N(q_1, 1) = \{q_2\}$ and $\delta_N(q_2, 1) = \emptyset$, so $\delta_D(\{q_0, q_1, q_2\}, 1) = \{q_0, q_1, q_2\}$

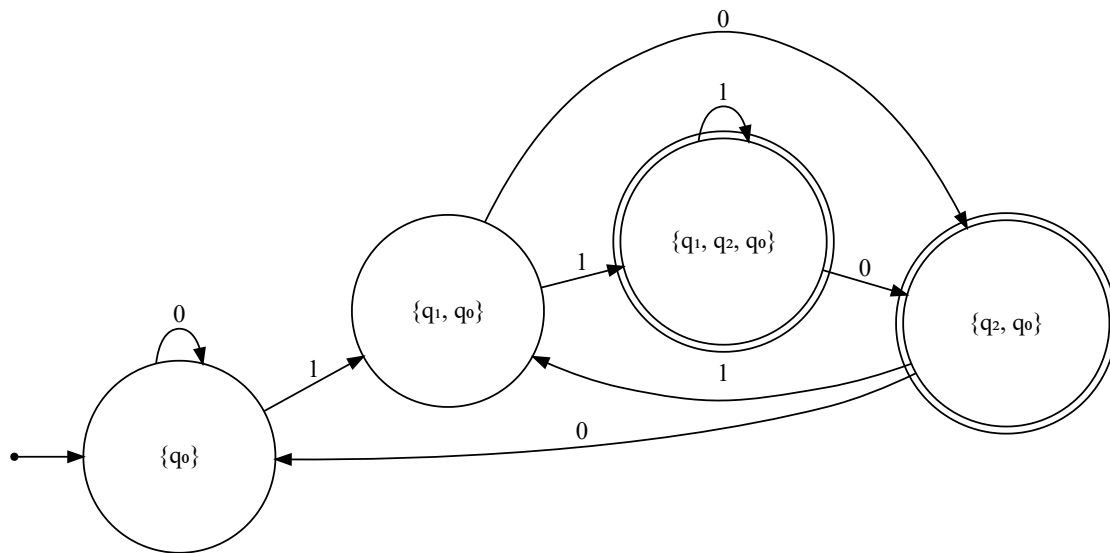


- No new states introduced, the DFA is complete (only needed 4 states instead of the potential 2^3)

```

G = DFA.from_nfa(nfa, retain_names=True, minify=False).diagram
for n in G.nodes():
    if 'circle' in n.attr['shape']:
        n.attr['width'] = 1.5
        n.attr['height'] = 1.5
        n.attr['fixedsize'] = True
        n.attr['label'] = str(n).replace(" ", {"", ""}, \n{"")
G.layout(prog='dot')
G

```



0 1 1 0
 {q₀} — 0 → {q₀} — 1 → {q₁, q₀} — 1 → {q₁, q₂, q₀} — 0 → {q₂, q₀} ✓

DFA → NFA: by definition

- An NFA is an automaton where, for each state/symbol pair, there are *arbitrary* transitions
- A DFA is an automaton where, for each state/symbol pair, there is *exactly one* transition
- So DFA are particular cases of NFAs by definition

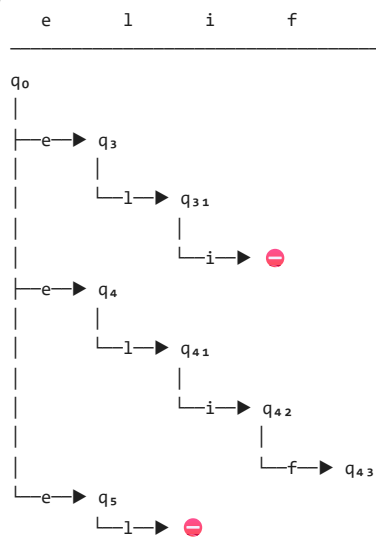
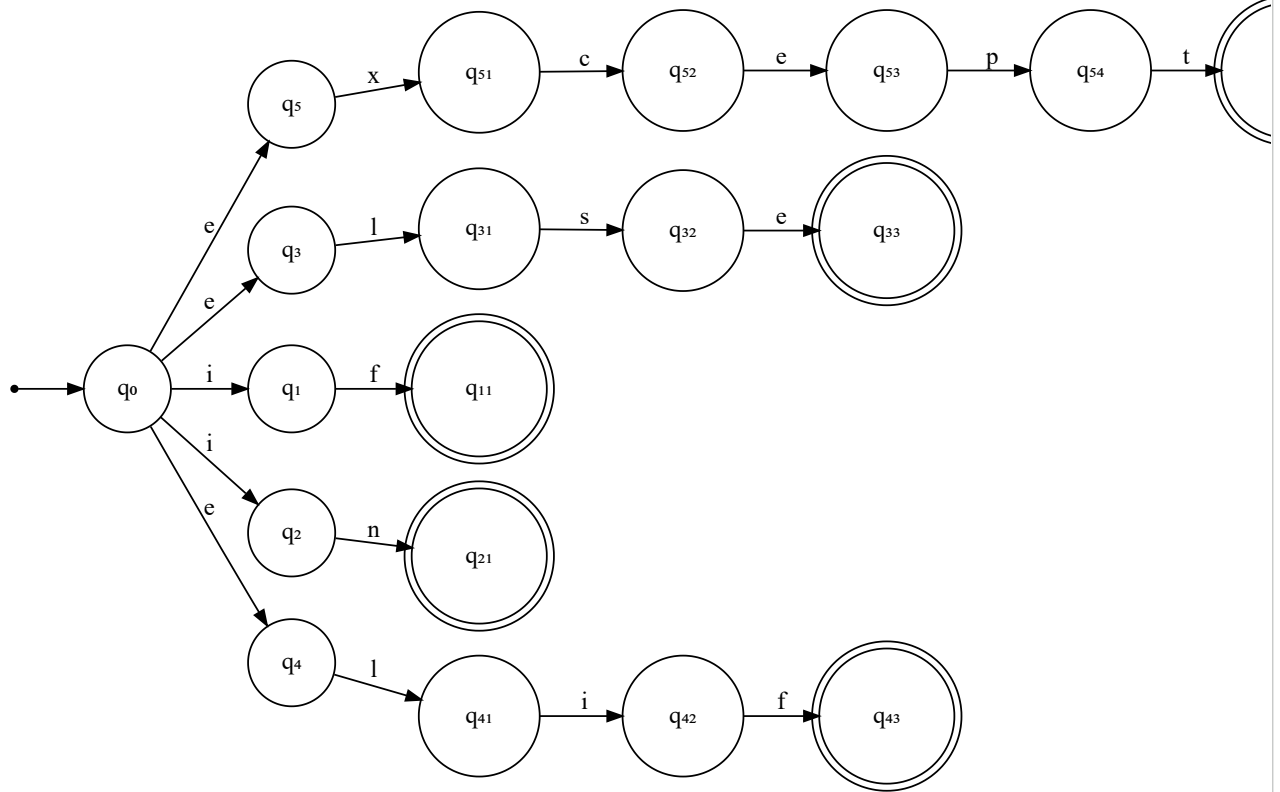
✓ Exercise: (Some) Python keywords

- Convert the following NFA into an equivalent DFA

```

import string
nfa = NFA(
    states={'q0', 'q1', 'q2', 'q3', 'q4', 'q5', 'q11', 'q21', 'q31', 'q32', 'q33', 'q41', 'q42', 'q43', 'q51', 'q52', 'q53', 'q54'},
    input_symbols=string.ascii_lowercase,
    transitions={
        'q0': { 'i' : {'q1', 'q2'}, 'e' : {'q3', 'q4', 'q5'} },
        'q1': { 'f' : {'q11'} },
        'q2': { 'n' : {'q21'} },
        'q3': { 'l' : {'q31'} },
        'q31': { 's' : {'q32'} },
        'q32': { 'e' : {'q33'} },
        'q4': { 'l' : {'q41'} },
        'q41': { 'i' : {'q42'} },
        'q42': { 'f' : {'q43'} },
        'q5': { 'x' : {'q51'} },
        'q51': { 'c' : {'q52'} },
        'q52': { 'e' : {'q53'} },
        'q53': { 'p' : {'q54'} },
        'q54': { 't' : {'q55'} },
    },
    initial_state='q0',
    final_states={'q11', 'q21', 'q33', 'q43', 'q55'} )
nfa.show_diagram(path=None)

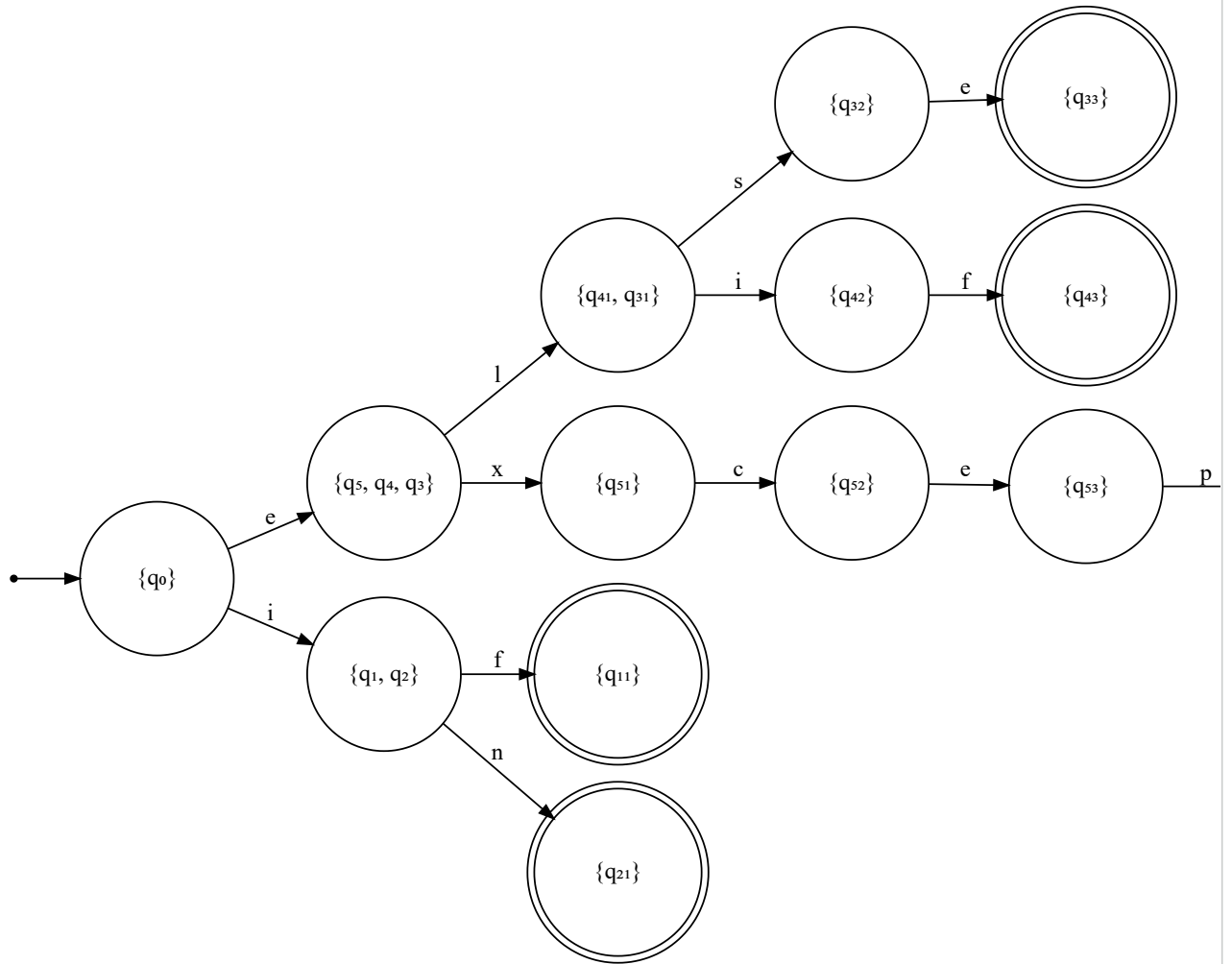
```



```

G = DFA.from_nfa(nfa, retain_names=True, minify=False).diagram
for n in G.nodes():
    if 'circle' in n.attr['shape']:
        n.attr['width'] = 1.25
        n.attr['height'] = 1.25
        n.attr['fixedsize'] = True
        n.attr['label'] = str(n).replace("}", {"", "\n{")
G.layout(prog='dot')
G

```



e l i f

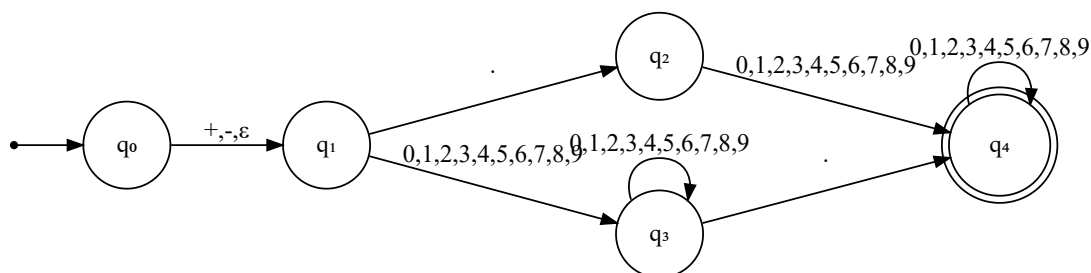
 $\{q_0\} \xrightarrow{e} \{q_3, q_4, q_5\} \xrightarrow{l} \{q_{31}, q_{41}\} \xrightarrow{i} \{q_{42}\} \xrightarrow{f} \{q_{43}\}$ ✓

✓ Non-deterministic finite automata with ε transitions

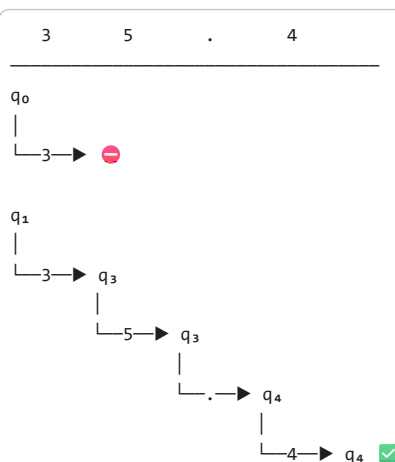
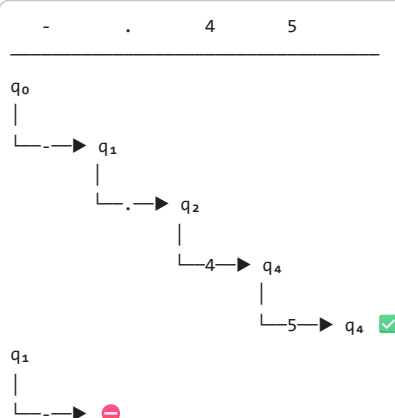
- ε transitions are spontaneous, they allow the state to change without consuming input symbols
- ε is not a symbol of the alphabet Σ , and does not appear in the words, it represents the absence of input
- A non-deterministic finite automaton with ε transitions (εNFA) is a tuple $(Q, \Sigma, \delta, q_i, F)$
 - Q is a finite set of states
 - Σ is the input alphabet
 - $\delta : Q \times \Sigma \cup \{\varepsilon\} \rightarrow \mathcal{P}(Q)$ is a transition function
 - $q_i \in Q$ is the initial state
 - $F \subseteq Q$ is a set of final (or accepting) states
- To reason about εNFAs, we need the ε -closure of a state, every state reachable through spontaneous transitions
 - base case: $q \in \varepsilon\text{-closure}(q)$
 - inductive case: if p is in $\varepsilon\text{-closure}(q)$, $\delta(p, \varepsilon) \subseteq \varepsilon\text{-closure}(q)$
 - we generalize this to apply to a set of states S as $\varepsilon\text{-closure}(S) = \bigcup_{q \in S} \varepsilon\text{-closure}(q)$
- The extended transition function $\hat{\delta} : Q \times \Sigma^* \rightarrow \mathcal{P}(Q)$ is then defined as
 - $\hat{\delta}(q, \varepsilon) = \varepsilon\text{-closure}(q)$
 - $\hat{\delta}(q, xa) = \varepsilon\text{-closure}\left(\bigcup_{p \in \hat{\delta}(q, x)} \delta(p, a)\right)$
- The accepting condition is the same as NFAs without ε transitions, since $\hat{\delta}$ applies the ε -closure
 - A word w is accepted iff $\hat{\delta}(q_i, w) \cap F \neq \emptyset$

Example: Floating point numbers

```
nfa = NFA(
    states={'q0', 'q1', 'q2', 'q3', 'q4'},
    input_symbols={'0', '1', '2', '3', '4', '5', '6', '7', '8', '9', '+', '-', '.'},
    transitions={
        'q0': { c : {'q1'} for c in { '+', '-', '.' } },
        'q1': { '.' : {'q2'} } | { c : {'q3'} for c in {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'} },
        'q2': { c : {'q4'} for c in {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'} },
        'q3': { '.' : {'q4'} } | { c : {'q3'} for c in {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'} },
        'q4': { c : {'q4'} for c in {'0', '1', '2', '3', '4', '5', '6', '7', '8', '9'} },
    },
    initial_state='q0',
    final_states={'q4'} )
nfa.show_diagram(path=None)
```



- $\varepsilon\text{-closure}(q_0) = \{q_0, q_1\}$
- $\varepsilon\text{-closure}(q_1) = \{q_1\}$
- $\varepsilon\text{-closure}(q_2) = \{q_2\}$
- $\varepsilon\text{-closure}(q_3) = \{q_3\}$
- $\varepsilon\text{-closure}(q_4) = \{q_4\}$



ε NFAs vs. NFAs vs. DFAs

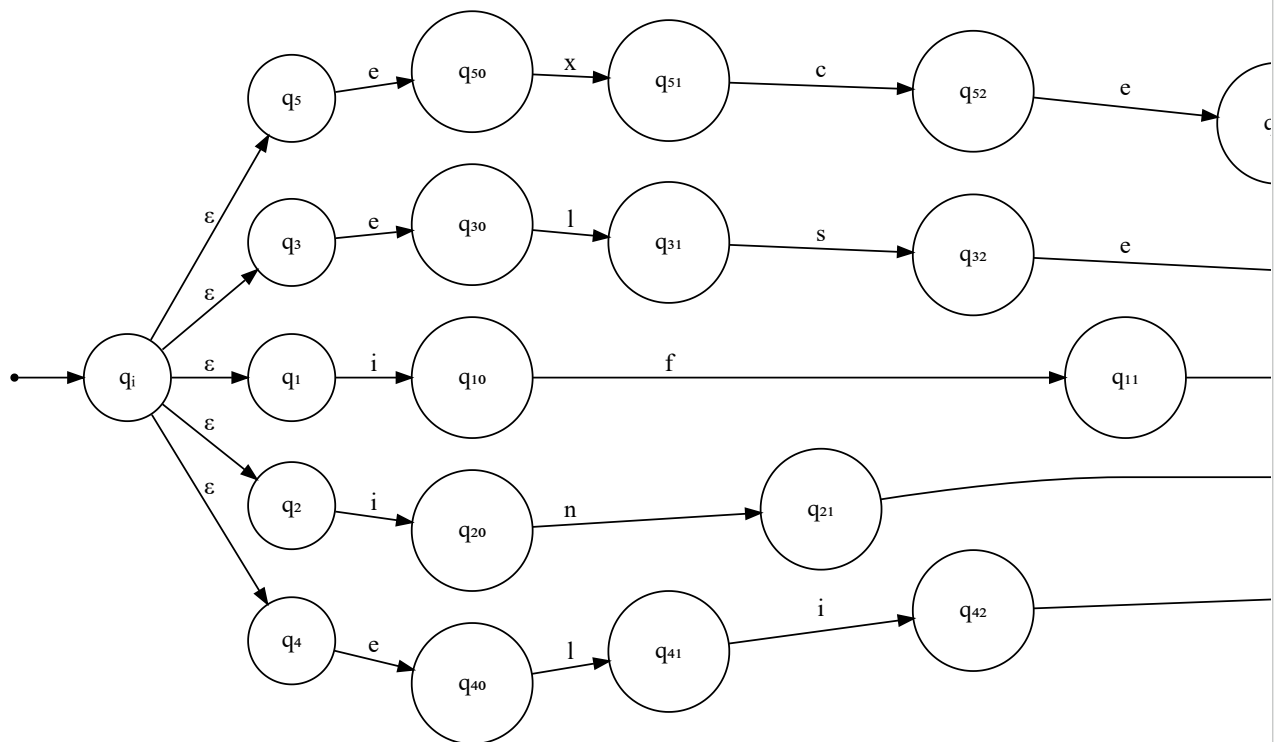
- ε NFAs are equivalent to NFAs and DFAs, they can recognize exactly the same languages
- ε NFAs allow us to more easily encode automata, that can be then converted into DFA for execution

- This is still done with the subset construction, but now considering the ϵ -closure

✓ Exercise: (Some) Python keywords revisited

- Convert the following ϵ NFA into a DFA

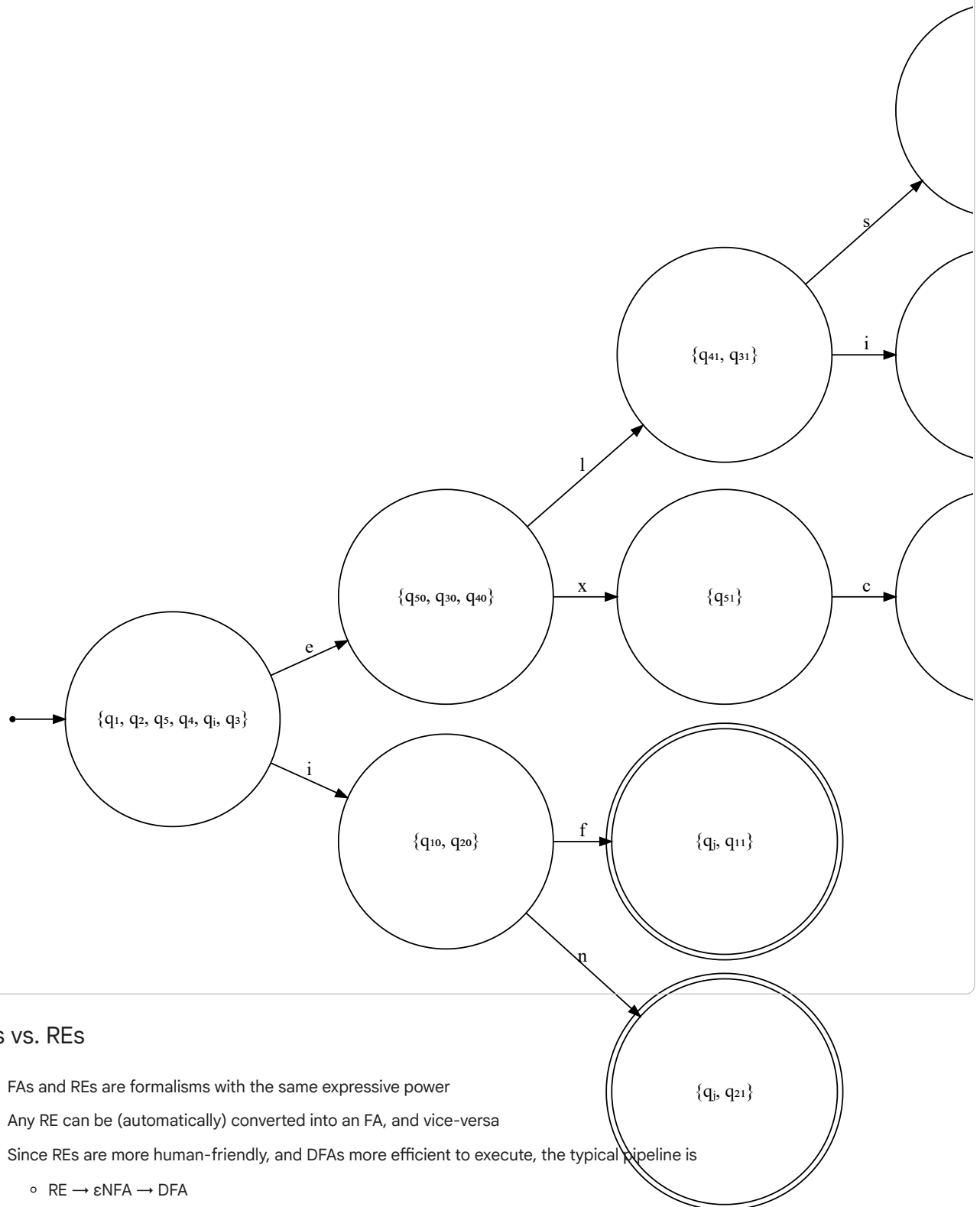
```
import string
nfa = NFA(
    states={'qi', 'q1', 'q2', 'q3', 'q4', 'q5', 'q10', 'q20', 'q30', 'q40', 'q50', 'q11', 'q21', 'q31', 'q32', 'q33', 'q41'},
    input_symbols=string.ascii_lowercase,
    transitions={
        'qi': { '': { 'q1', 'q2', 'q3', 'q4', 'q5' } },
        'q1': { 'i' : { 'q10' } },
        'q2': { 'i' : { 'q20' } },
        'q3': { 'e' : { 'q30' } },
        'q4': { 'e' : { 'q40' } },
        'q5': { 'e' : { 'q50' } },
        'q10': { 'f' : { 'q11' } },
        'q20': { 'n' : { 'q21' } },
        'q30': { 'l' : { 'q31' } },
        'q31': { 's' : { 'q32' } },
        'q32': { 'e' : { 'q33' } },
        'q40': { 'l' : { 'q41' } },
        'q41': { 'i' : { 'q42' } },
        'q42': { 'f' : { 'q43' } },
        'q50': { 'x' : { 'q51' } },
        'q51': { 'c' : { 'q52' } },
        'q52': { 'e' : { 'q53' } },
        'q53': { 'p' : { 'q54' } },
        'q54': { 't' : { 'q55' } },
        'q11': { '' : { 'qj' } },
        'q21': { '' : { 'qj' } },
        'q33': { '' : { 'qj' } },
        'q43': { '' : { 'qj' } },
        'q55': { '' : { 'qj' } },
    },
    initial_state='qi',
    final_states={'qj'} )
nfa.show_diagram(path=None)
```



```

G = DFA.from_nfa(nfa, retain_names=True, minify=False).diagram
for n in G.nodes():
    if 'circle' in n.attr['shape']:
        n.attr['width'] = 2.2
        n.attr['height'] = 2.2
        n.attr['fixedsize'] = True
        n.attr['label'] = str(n).replace("}", {"", "\n{")
G.layout(prog='dot')
G

```



✓ FAs vs. REs

- FAs and REs are formalisms with the same expressive power
- Any RE can be (automatically) converted into an FA, and vice-versa
- Since REs are more human-friendly, and DFAs more efficient to execute, the typical pipeline is
 - RE → εNFA → DFA

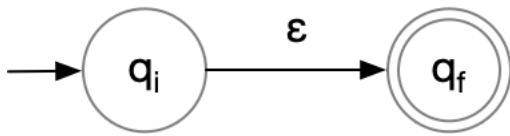
RE → εNFA: Thompson's construction

- Thompson's construction acts inductively on an RE to convert it into an εNFA
- Each step produces an εNFA with exactly one initial state and one accepting state
- Base cases:

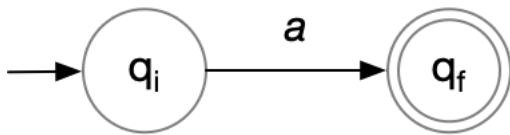
- $\emptyset$



- ϵ

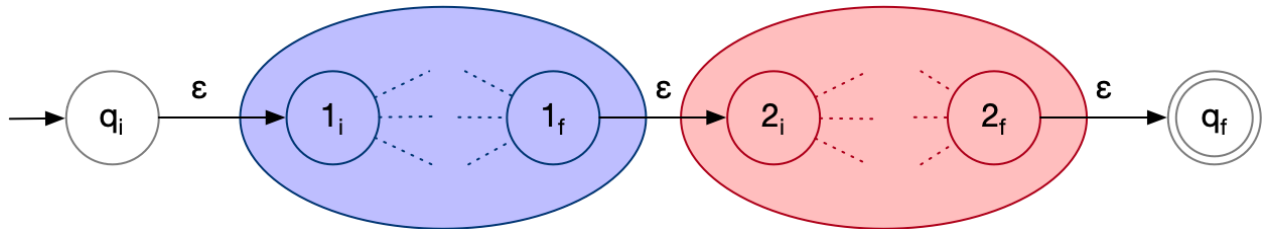


- $a \in \Sigma$

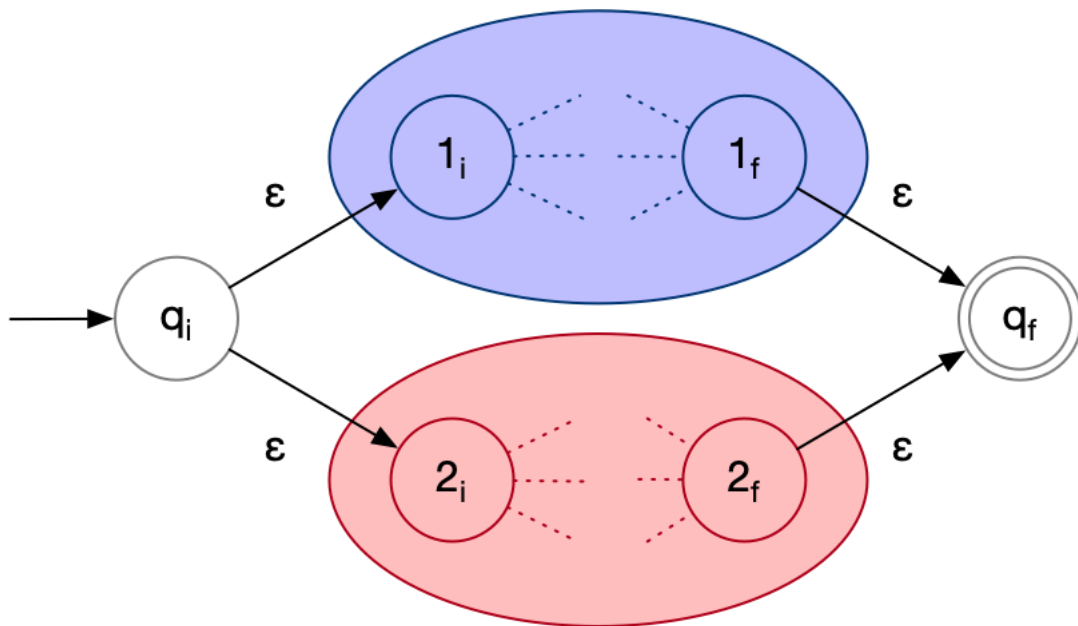


- Inductive steps, for re_k with initial state k_i and k_f

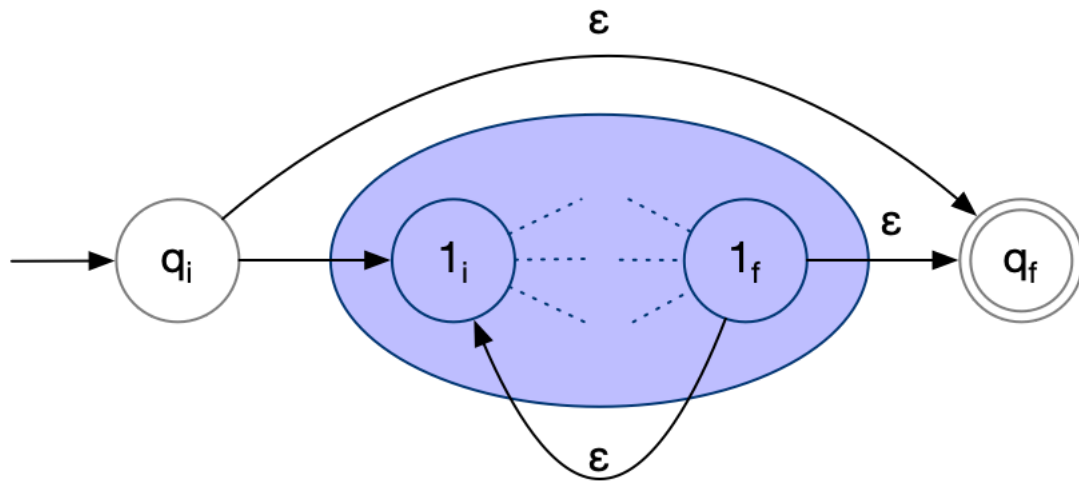
- re_1re_2



- $re_1|re_2$



- re_1^*

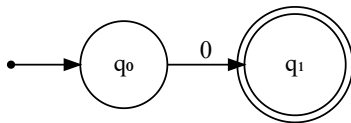


- Example: Bitstrings with at least one 1 revisited

$0^*1(0|1)^*$

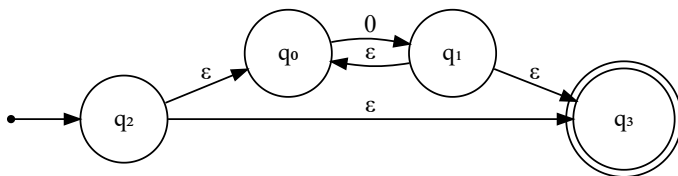
0

```
nfa = NFA(
    states={'q0', 'q1'},
    input_symbols={'0', '1'},
    transitions={
        'q0': {'0': {'q1'}} },
    initial_state='q0',
    final_states={'q1'} )
nfa.show_diagram(path=None)
```



0^*

```
nfa = NFA(
    states={'q0', 'q1', 'q2', 'q3'},
    input_symbols={'0', '1'},
    transitions={
        'q0': {'0': {'q1'}},
        'q2': {'': {'q0', 'q3'}},
        'q1': {'': {'q0', 'q3'}}},
    initial_state='q2',
    final_states={'q3'} )
nfa.show_diagram(path=None)
```



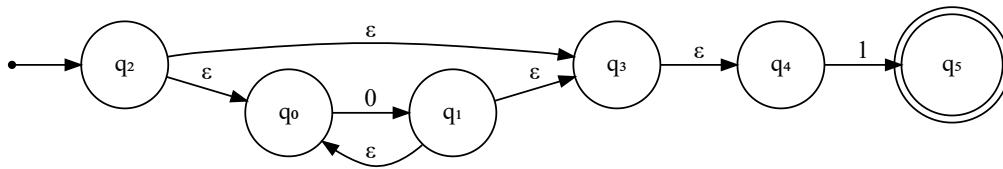
0^*1

```
nfa = NFA(
    states={'q0', 'q1', 'q2', 'q3', 'q4', 'q5'},
    input_symbols={'0', '1'},
    transitions={
        'q0': {'0': {'q1'}},
        'q2': {'': {'q0', 'q3'}},
        'q1': {'': {'q0', 'q3'}},
        'q3': {'': {'q4'}},
        'q4': {'1': {'q5'}}},
    ),
```

```

initial_state='q2',
final_states={'q5'} )
nfa.show_diagram(path=None)

```

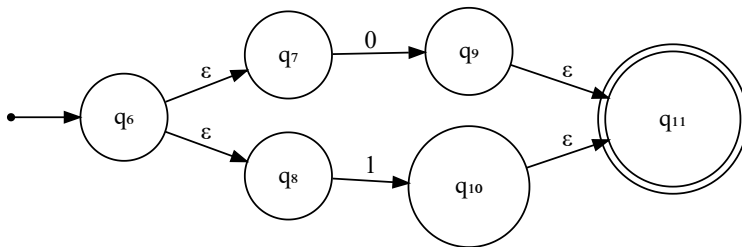


$0|1$

```

nfa = NFA(
    states={'q6', 'q7', 'q8', 'q9', 'q10', 'q11'},
    input_symbols={'0', '1'},
    transitions={
        'q6': {'': {'q7', 'q8'}},
        'q7': {'0': {'q9'}},
        'q8': {'1': {'q10'}},
        'q9': {'': {'q11'}},
        'q10': {'': {'q11'}},
    },
    initial_state='q6',
    final_states={'q11'} )
nfa.show_diagram(path=None)

```

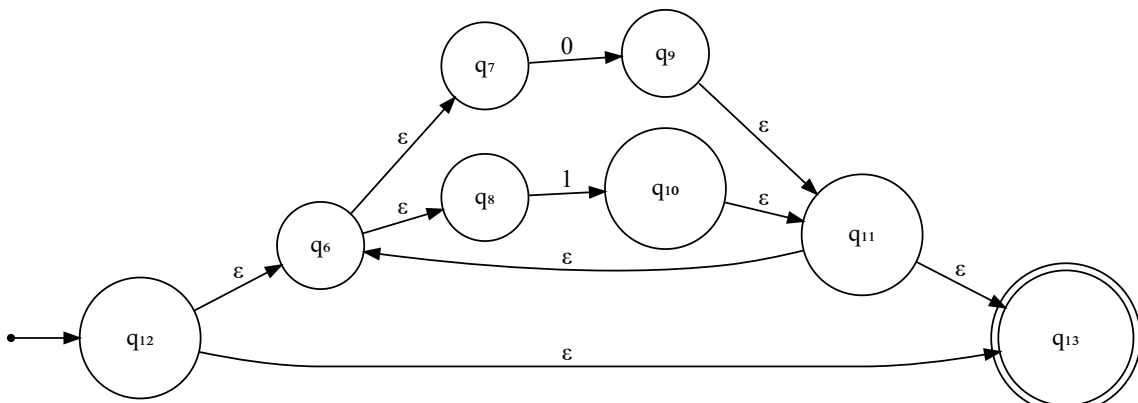


$(0|1)^*$

```

nfa = NFA(
    states={'q6', 'q7', 'q8', 'q9', 'q10', 'q11', 'q12', 'q13'},
    input_symbols={'0', '1'},
    transitions={
        'q6': {'': {'q7', 'q8'}},
        'q7': {'0': {'q9'}},
        'q8': {'1': {'q10'}},
        'q9': {'': {'q11'}},
        'q10': {'': {'q11'}},
        'q11': {'': {'q13', 'q6'}},
        'q12': {'': {'q6', 'q13'}},
    },
    initial_state='q12',
    final_states={'q13'} )
nfa.show_diagram(path=None)

```

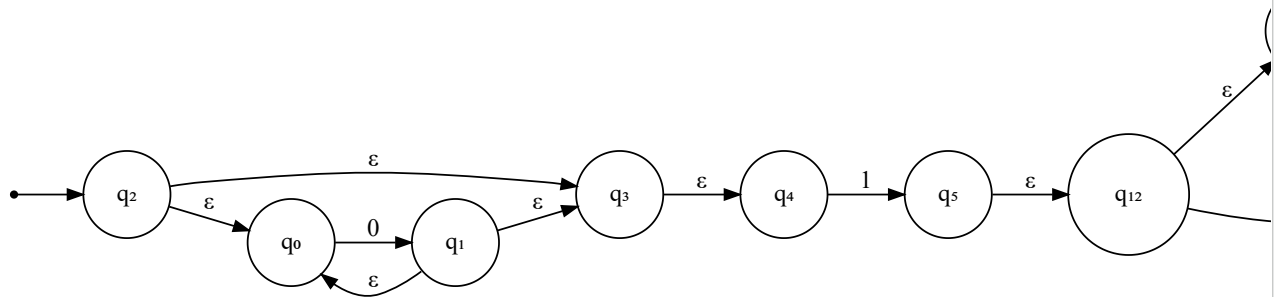


$0^*1(0|1)^*$

```

nfa = NFA(
    states={'q0', 'q1', 'q2', 'q3', 'q4', 'q5', 'q6', 'q7', 'q8', 'q9', 'q10', 'q11', 'q12', 'q13'},
    input_symbols={'0', '1'},
    transitions={
        'q0': {'0' : {'q1'}},
        'q2': {'' : {'q0', 'q3'}},
        'q1': {'' : {'q0', 'q3'}},
        'q3': {'' : {'q4'}},
        'q4': {'1' : {'q5'}},
        'q6': {'' : {'q7', 'q8'}},
        'q7': {'0' : {'q9'}},
        'q8': {'1' : {'q10'}},
        'q9': {'' : {'q11'}},
        'q10': {'' : {'q11'}},
        'q11': {'' : {'q13', 'q6'}},
        'q12': {'' : {'q6', 'q13'}},
        'q5': {'' : {'q12'}}},
    initial_state='q2',
    final_states={'q13'} )
nfa.show_diagram(path=None)

```



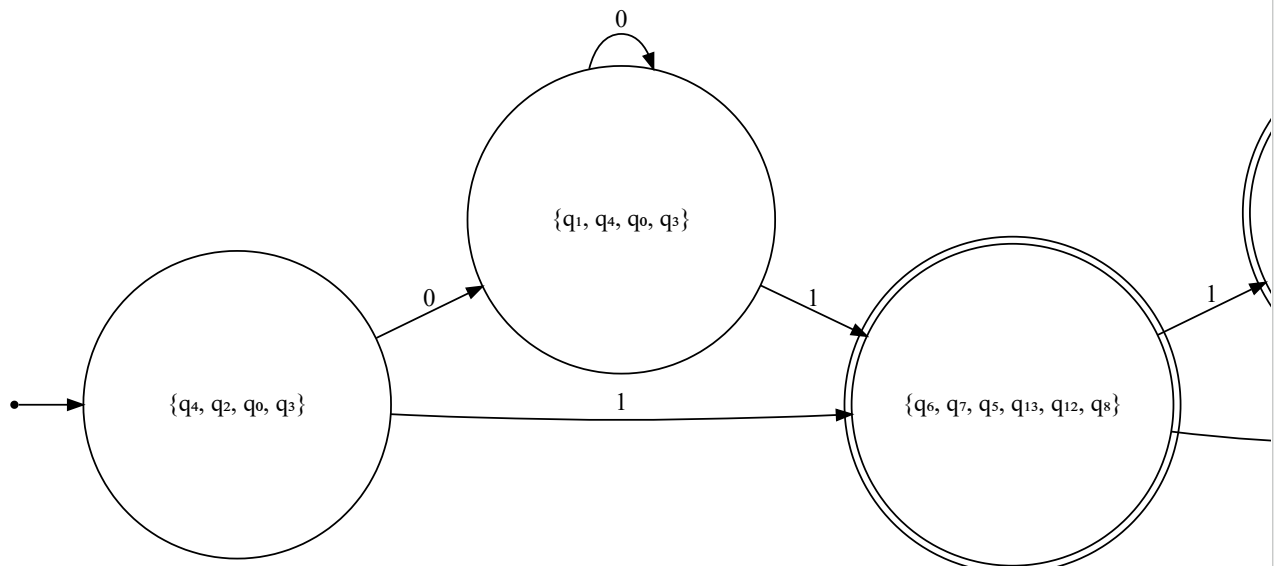
```

dfa = DFA.from_nfa(nfa,minify=False,retain_names=True)
G = dfa.diagram
for n in G.nodes():
    if 'circle' in n.attr['shape']:
        n.attr['width'] = 2.4
        n.attr['height'] = 2.4
        n.attr['fixedsize'] = True
        n.attr['label'] = str(n).replace("{", "{",",",\n{"")
G.layout(prog='dot')
G

```

/usr/local/lib/python3.12/dist-packages/pygraphviz/agraph.py:1403: RuntimeWarning: Warning: node '{q6, q7, q5, q13, q12, q8}'
Warning: node '{q13, q7, q10, q6, q11, q8}', graph '' size too small for label
Warning: node '{q6, q7, q9, q13, q11, q8}', graph '' size too small for label

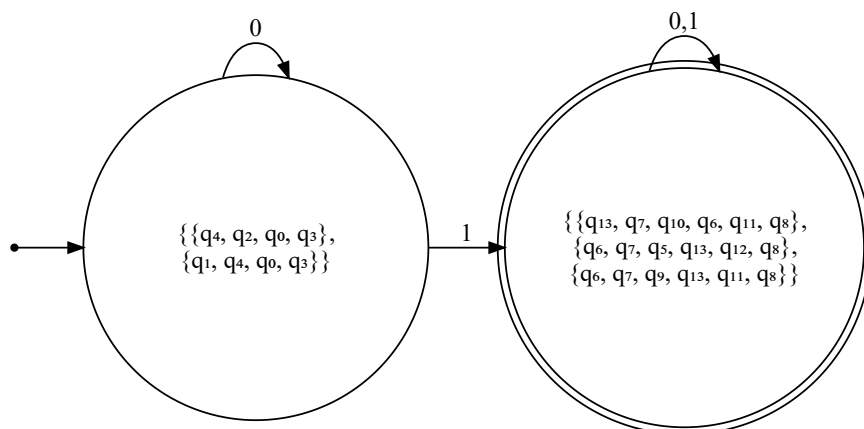
```
warnings.warn(b"".join(errors).decode(self.encoding), RuntimeWarning)
```



Minimal DFAs

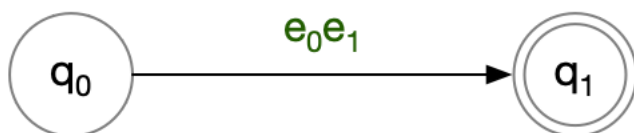
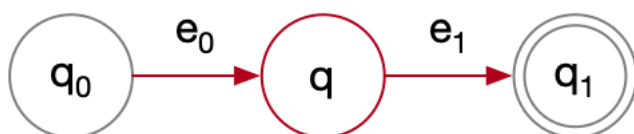
- DFAs (but not NFAs!) have a unique, **minimized** version, that can be automatically calculated
- It works by identifying *equivalent* states and merging them
- This allows more efficient implementations
- It also allows to test automata equivalence: two automata are equivalent if their minimized deterministic versions are isomorphic

```
G = dfa.minify(retain_names=True).diagram
for n in G.nodes():
    if 'circle' in n.attr['shape']:
        n.attr['width'] = 2.7
        n.attr['height'] = 2.7
        n.attr['fixedsize'] = True
        n.attr['label'] = str(n).replace(" ", {"", ""}, \n{"")
G.layout(prog='dot')
G
```



ϵ NFA $\rightarrow$ RE: State elimination

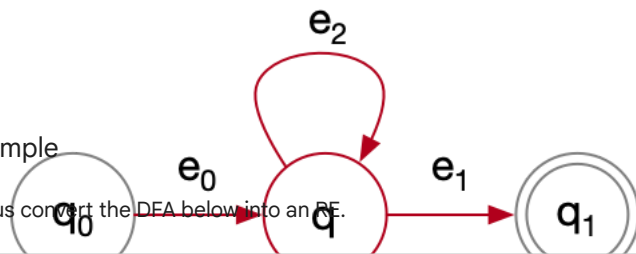
- Main idea: each transition in the FA will represent an RE instead of a single symbol
- Eliminate states from the FA one by one, updating the transitions to a RE that represents traversing the eliminated state
- Essentially, if I'm eliminating q , if I have an incoming edge from q_0 through expression e_0 , and an outgoing edge to q_1 with expression e_1 , there is a new edge between q_0 and q_1 with expression e_0e_1



- If the state being eliminated has a self-loop, it results in a Kleene closure inside the new transition

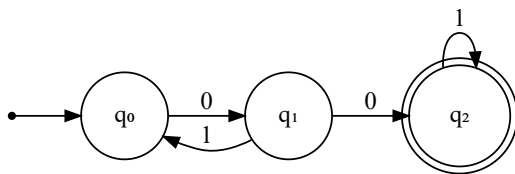
✓ Example

Let us convert the DFA below into an RE.

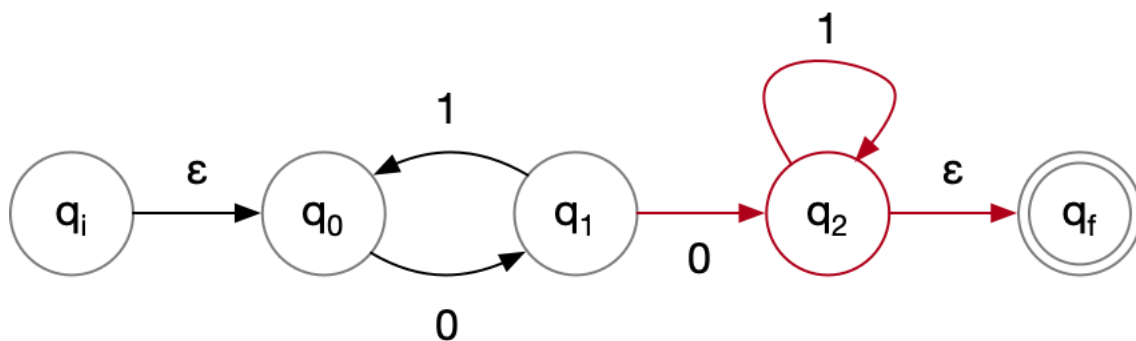


```

NFA(
  states={'q0', 'q1', 'q2'},
  input_symbols={'0', '1'},
  transitions={
    'q0': {'0' : {'q1'}},
    'q1': {'0' : {'q2'}, '1' : {'q0'}},
    'q2': {'1' : {'q2'}}},
  initial_state='q0',
  final_states={'q2'} ).show_diagram(path=None)
  
```



- Let's start to normalize the DFA (initial state with only outgoing edges, single accepting state with only incoming edges)
- This results in a ϵ NFA



- Let's start by removing for instance q_0 and its edges (marked red)